



UNIVERSITÀ DEGLI STUDI DI NAPOLI
FEDERICO II

*DOCUMENTO DI SPECIFICA DEI REQUISITI SOFTWARE
"BUGBOARD26"*

CORSO DI INGEGNERIA DEL SOFTWARE

Gabriele Letizia - N86005267

Fabio Iannicelli - N86005383

ANNO ACCADEMICO 2025/26

Indice

1	Introduzione	3
1.1	Scopo	3
1.2	Obiettivi	3
1.3	Definizioni, acronimi e abbreviazioni	4
1.3.1	Definizioni	4
2	Caratterizzazione degli Utenti	8
3	Modellazione tramite Use Case Diagram	9
4	Requisiti Specifici	10
4.1	Requisiti Funzionali	10
4.2	Requisiti non-funzionali e di sistema	11
5	Specifica dei Casi d'Uso	13
5.1	UC1: Segnalazione di una Issue	13
5.2	UC2: Archiviazione di una issue	14
5.3	Mock-Up UC1: Segnalazione di una Issue	15
5.4	Mock-Up UC2: Archiviazione di una Issue	15
6	Architettura Proposta	16
6.1	Deployment Diagram	17
6.2	Back-end (Server)	18
6.3	Front-end (Client)	18
6.4	Criteri di Design e Astrazione	18
7	Scelte Tecnologiche	19
7.1	Tecnologie Back-end	19
7.2	Tecnologie Front-end	19
7.3	Strumenti di Sviluppo e Modellazione	19
8	Piano e Strategie di Testing	21
8.1	Test Plan: Assegnazione Issue	21
8.2	Strategie di Testing	22
8.2.1	Analisi Metodo cambio password	22
8.2.2	Analisi metodo assegnazione di una issue ad un utente	23
8.2.3	Analisi metodo login	24
8.3	Codice Sorgente dei Test	25
8.3.1	Test: Cambio Password	25
8.3.2	Test: Login	27
8.3.3	Test: Assegnazione issue ad un utente	29

1 Introduzione

1.1 Scopo

Questo documento ha lo scopo di definire le specifiche dei requisiti per una piattaforma dedicata alla gestione collaborativa di issue in progetti software. Il documento si rivolge a tutti gli utenti del sistema, illustrandone le relative specifiche software e i vincoli.

1.2 Obiettivi

Il prodotto software che si intende realizzare prende il nome di “BugBoard26”. Il sistema dovrà offrire le seguenti funzionalità:

- 1) Gestire l'autenticazione degli utenti, consentendo a ciascuno di accedere alle sole funzionalità che gli competono.
- 2) Consentire agli amministratori di creare nuove utenze, potendone assegnare il ruolo.
- 3) Permettere utenti interni e amministratori di segnalare nuove issue.
- 4) Fornire una vista riepilogativa delle issue, includendo la possibilità di filtrare o ordinare i risultati.
- 5) Fornire agli amministratori una dashboard contenente informazioni aggregate sulle issue.
- 6) Consentire agli amministratori di archiviare le issue.
- 7) Fornire una sezione dove è possibile visualizzare tutte le issue archiviate.
- 8) Permettere agli amministratori di contrassegnare una issue come duplicata di un'altra, comportandone l'eliminazione immediata.
- 9) Consentire visualizzazione delle issue da parte di utenti esterni con appositi account creati dagli amministratori.

Mentre non è prevista l'implementazione delle seguenti funzionalità:

- 1) Un sistema automatico di notifiche.
- 2) L'Esportazione dei dati, come ad esempio dell'elenco delle issue.
- 3) Sistema di messaggistica tra utenti.
- 4) Creazione di ulteriori ruoli rispetto ai tre previsti ("Utente interno", "Amministratore" e "Utente esterno").
- 5) Un sistema di registrazione alla piattaforma da parte degli utenti.

BugBoard26 ha come obiettivo principale quello di affiancare team di sviluppo, fungendo da piattaforma di segnalazione di problemi relativi a un progetto e di monitorarne lo stato, ottimizzando così la collaborazione operativa tra i membri del team.

1.3 Definizioni, acronimi e abbreviazioni

1.3.1 Definizioni

- **Issue** – Indica un elemento che rappresenta qualcosa da risolvere o da gestire all'interno di un progetto.
- **Amministratore** – Indica un ruolo "speciale" di utente, che ha uno scopo di gestione, come creare nuove utenze e la gestione degli account.
- **Utente interno** – Indica un'utenza "normale", che può usufruire di tutte le funzionalità che gli competono.
- **Utenti esterni** – Persone che non fanno parte del team interno di sviluppo o gestione del progetto, ma che hanno interesse a seguirne l'andamento.
- **Bug** – È una issue per segnalare un malfunzionamento.
- **Ruolo** – Insieme di permessi che determina le funzionalità e le operazioni accessibili a un utente all'interno del sistema.
- **Question** – È una issue usata per richieste di chiarimenti.
- **Feature** – È una issue per indicare la richiesta o il suggerimento di nuove funzionalità.
- **Documentation** – È una issue per segnalare problemi relativi alla documentazione
- **Team** – È un gruppo di persone che collaborano insieme per portare a termine attività o progetti.
- **Dashboard** – È un'interfaccia che mostra in modo chiaro e riassuntivo le principali informazioni sulle issue del sistema.
- **BugBoard26** – È il nome usato per identificare il software.
- **Software** – Insieme di programmi e istruzioni che permettono di svolgere determinate funzioni o compiti.
- **Modalità read-only** - Modalità di accesso che permette la sola visualizzazione delle issue per gli utenti esterni, senza possibilità di modificarle.
- **Stato della Issue** – Indica la fase del ciclo di vita di una issue. Una issue può trovarsi negli stati "Todo", "Assegnato", "Risolto" e "Archiviato".
- **E-mail** - Strumento informatico per l'invio della posta elettronica.
- **Password** – Insieme di caratteri alfanumerici che abilitano l'accesso ad aree riservate a determinati utenti.
- **Eliminazione** – Operazione in cui una issue viene rimossa definitivamente dal sistema.

- **Archiviazione** – Operazione in cui una issue viene rimossa dall’elenco principale rimanendo consultabile nell’apposita sezione delle issue archiviate.
- **Interfaccia utente** – È la parte di software con cui l’utente interagisce direttamente. Include tutti gli elementi visivi e di controllo, come menù, pulsanti, finestre, icone e moduli, che permettono di usare il programma in modo semplice ed efficace.
- **Autenticazione** - È il processo tramite il quale un sistema verifica l’identità di un utente prima di permettergli l’accesso. Serve a garantire che solo persone autorizzate possano entrare in un’applicazione, un sito o un servizio.
- **Use Case Diagram** - È un tipo di diagramma UML che serve a rappresentare graficamente le funzionalità principali di un sistema viste dal punto di vista dell’utente. Mostra chi (gli attori) interagisce con il sistema e quali azioni (i casi d’uso) può compiere.
- **Autorizzazione** – Processo mediante il quale il sistema determina quali risorse e funzionalità possono essere utilizzate da un utente autenticato, sulla base dei permessi associati al suo ruolo.
- **UML (Unified Modeling Language)** - Linguaggio di modellazione standard usato per rappresentare graficamente la struttura, il comportamento e le interazioni di un sistema software.
- **Account** – Insieme di informazioni associate a un utente che ne consentono l’identificazione e l’accesso al sistema.
- **Todo** – Indica un’attività ancora da completare o un promemoria, in questo caso, la gestione di una issue.
- **Back-end** - È la componente del sistema responsabile della gestione della logica applicativa e dei dati. È separata dal Front-end e ne supporta le funzionalità attraverso API accessibili via rete.
- **Front-end** - È la componente che rappresenta l’interfaccia utente dell’applicazione. Comunica con il Back-end esclusivamente tramite chiamate di rete alle API esposte e può essere eseguita separatamente da esso.
- **Template di A. Cockburn** - È una tabella utilizzata per fornire la descrizione testuale dettagliata dei casi d’uso all’interno del documento di specifica.
- **Tool di CASE** - Strumenti software, come Visual Paradigm, che vengono utilizzati per lo svolgimento delle attività di progettazione e modellazione UML.
- **API REST (Application Programming Interface)** - Sono le interfacce di programmazione accessibili via rete esposte dal back-end, attraverso le quali il front-end comunica per scambiare dati.
- **Mock-up** - È un prototipo visuale utilizzato nella documentazione per rappresentare graficamente la progettazione delle interfacce utente relative ai casi d’uso.

- **MBaaS (Mobile Backend as a Service)** - Modello di cloud computing in cui l'infrastruttura e la logica di back-end vengono fornite e gestite interamente da terze parti.
- **Docker** - Tecnologia che permette di creare dei container, cioè impacchettare un'applicazione e tutte le sue dipendenze, garantendo che il software funzioni in modo identico in qualsiasi ambiente.
- **Client-Server** - Architettura di rete in cui un componente (Client) richiede servizi o delle risorse, e un'altra componente (Server) li elabora e li fornisce.
- **Framework** - Struttura di supporto su cui un software può essere progettato e realizzato, fornisce funzionalità preimpostate (ad esempio Spring Boot).
- **Rollback** - Operazione che consente di riportare una transazione o un sistema a uno stato precedente, annullando le modifiche effettuate nell'ambito dell'operazione.
- **Sistema di Versioning** - Strumento software (come ad esempio Git) che traccia, storicizza e gestisce tutte le modifiche apportate al codice sorgente nel corso del tempo da un team di sviluppo.
- **JWT (JSON Web Token)** - Formato standard per la rappresentazione di informazioni sotto forma di token, utilizzato per trasmettere in modo strutturato informazioni tra le parti di un sistema, ad esempio nell'ambito dell'autenticazione e dell'autorizzazione.
- **SonarQube** - Strumento per l'ispezione della qualità del codice, utilizzato per eseguire analisi e rilevare automaticamente bug, cattive pratiche di programmazione, vulnerabilità e codice duplicato.
- **Dependency Injection** - Principio in cui le dipendenze di un oggetto (le altre classi di cui ha bisogno per funzionare) gli vengono fornite dall'esterno, favorendo un forte disaccoppiamento del codice.
- **Object-Oriented (OO)** - Paradigma di programmazione basato sul concetto di oggetti, cioè strutture che incapsulano al loro interno informazioni (attributi) e i comportamenti (metodi).
- **SOLID** - Acronimo che raggruppa cinque principi fondamentali (S: Single responsibility, O: Open-Closed, L: Liskov substitution, I: Interface segregation, D: Dependency inversion) del paradigma OO, volti a rendere il software più flessibile alle modifiche e più manutenibile.
- **Java** - Linguaggio di programmazione orientato agli oggetti, ad alto livello e fortemente tipizzato. È stato progettato per essere indipendente dalla piattaforma hardware di esecuzione.
- **JavaFX** - Framework per la realizzazione di interfacce grafiche di applicazioni desktop sviluppate in Java.

Documento di specifica dei requisiti software

- **PostgreSQL** - Sistema di gestione di database relazionali, utilizzato per la memorizzazione e la gestione persistente dei dati del sistema.
- **Architettura a Livelli** - Architettura in cui il software è suddiviso in strati separati e indipendenti, ognuno con una responsabilità specifica.
- **Model** - Strato dell'architettura che rappresenta le entità fondamentali del sistema.
- **Repository** - Componente dell'architettura responsabile dell'accesso ai dati persistenti e dell'interazione con il database, isolando tale responsabilità dalla logica applicativa.
- **Service** - Componente dell'architettura intermedio che gestisce l'intera logica di business e le regole applicative, operando tra Controller e i Repository.
- **Controller** - Componente dell'architettura che gestisce le richieste provenienti dal client attraverso le API, coordinando l'interazione con i livelli sottostanti e restituendo le relative risposte.
- **FXML** - Linguaggio di markup basato su XML utilizzato in JavaFX per definire la struttura e l'aspetto dell'interfaccia grafica (View) in modo separato dal codice logico (Controller).
- **Linguaggio di Markup (o di marcatura)** - Sistema di regole e convenzioni per definire la struttura, la formattazione e la disposizione degli elementi in un documento o in un'interfaccia (es. XML, HTML, FXML).
- **Classi Helper** - Classi di supporto progettate per contenere metodi e funzioni generiche e riutilizzabili da diverse parti dell'applicazione, al fine di evitare duplicazioni di codice.
- **Stateless** - Architettura nella quale il server non mantiene lo stato della sessione del client tra richieste successive; ogni richiesta contiene le informazioni necessarie per essere elaborata.
- **Spring Boot** - Framework basato su Java utilizzato per semplificare lo sviluppo e la configurazione di applicazioni server, in particolare per la realizzazione di API REST.
- **Spring Security** - Framework utilizzato per gestire i meccanismi di autenticazione e autorizzazione dell'applicazione.
- **Spring Data JPA** - Modulo di Spring che semplifica l'accesso ai dati persistenti attraverso JPA e i repository.
- **JPA** - Specifica Java che definisce un'interfaccia standard per il mapping tra oggetti Java ed entità di un database relazionale.
- **Hibernate** - Framework utilizzato come implementazione di JPA per gestire la persistenza degli oggetti Java nel database.
- **REST (Representational State Transfer)** - Stile architetturale utilizzato per progettare servizi accessibili attraverso la rete, tipicamente mediante il protocollo HTTP.

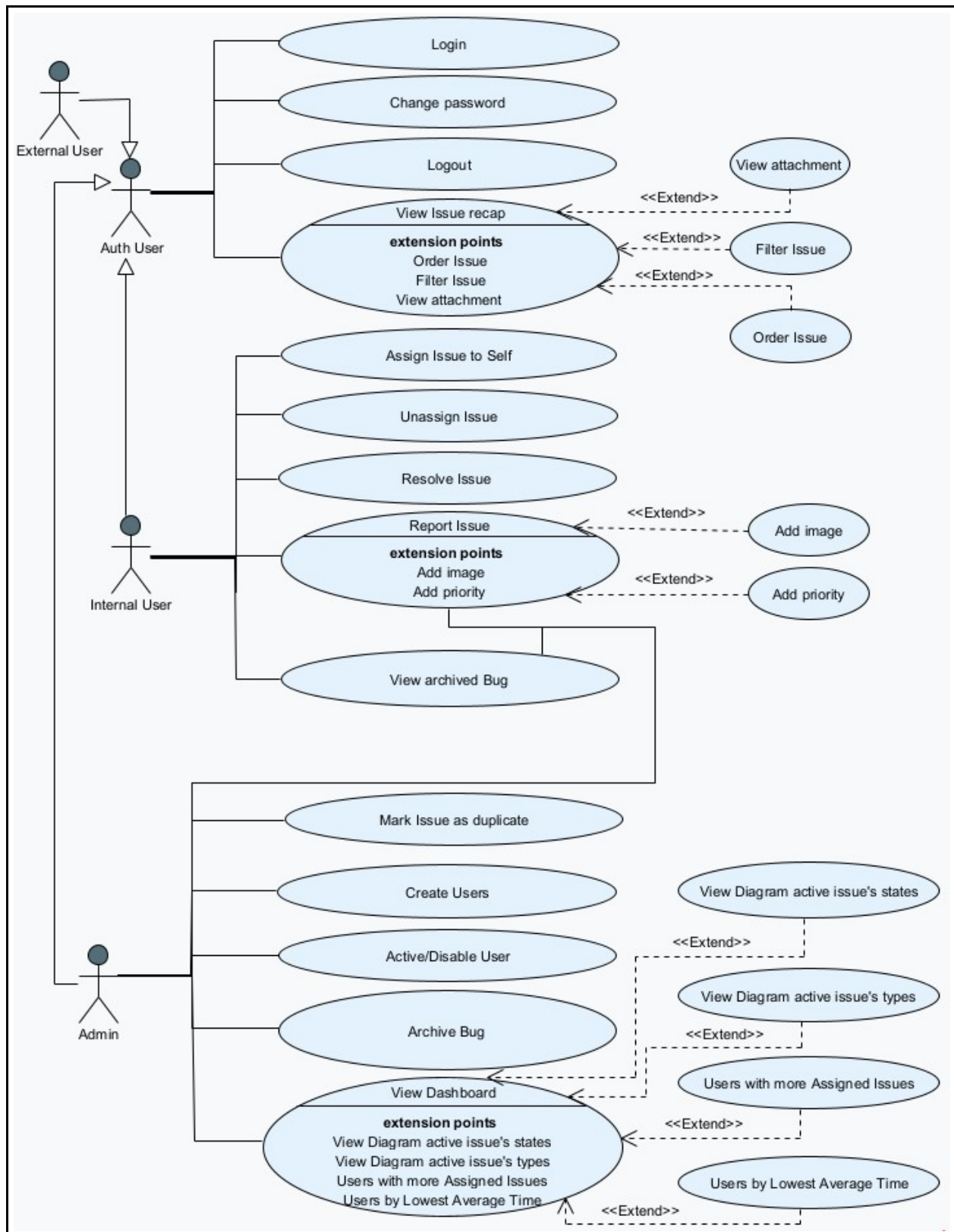
- **JSON (JavaScript Object Notation)** - Formato testuale utilizzato per rappresentare e scambiare dati strutturati tra applicazioni.
- **HTTP (HyperText Transfer Protocol)** - Protocollo di comunicazione utilizzato per lo scambio di richieste e risposte tra Client e Server.
- **API (Application Programming Interface)** - Insieme di interfacce e operazioni attraverso cui un componente software può interagire con un altro componente.
- **Transazione** - Sequenza di operazioni eseguite come un'unica unità logica, il cui risultato deve rispettare le proprietà di consistenza previste dal sistema.

2 Caratterizzazione degli Utenti

Gli utenti di BugBoard26 devono avere una conoscenza base dell'uso del computer. Ogni utente può accedere esclusivamente alle funzionalità ed ai dati previsti dal proprio ruolo. Gli utenti di BugBoard26 sono:

- **Amministratore** - È un utente dotato di privilegi amministrativi, può visualizzare dati aggregati e statistiche attraverso una Dashboard di controllo e svolge le principali attività di gestione del sistema.
- **Utente interno** - È un membro del team di sviluppo che accede al sistema tramite credenziali personali (e-mail e password). Non dispone di permessi amministrativi e può operare solo sulle issue di propria competenza.
- **Utente esterno** - È un soggetto non appartenente al team interno di sviluppo, ma interessato a consultare lo stato delle issue, come ad esempio uno stakeholder o un cliente. Gli account di questo tipo sono creati dagli amministratori e consentono la consultazione delle issue in modalità read-only, favorendo la trasparenza e il monitoraggio da parte di soggetti esterni all'organizzazione.

3 Modellazione tramite Use Case Diagram



4 Requisiti Specifici

4.1 Requisiti Funzionali

BugBoard26 prevede le seguenti funzionalità:

- **RF-01 - Autenticazione:** All'avvio di BugBoard26, l'utente deve autenticarsi mediante e-mail e password. Il sistema verifica la validità delle credenziali e, in caso di autenticazione avvenuta con successo, consente l'accesso alle funzionalità previste dal ruolo associato all'account.
- **RF-02 - Cambio password:** Una volta autenticato, ogni utente deve avere la possibilità di aggiornare la password del proprio account.
- **RF-03 - Logout:** Ogni utente autenticato deve avere la possibilità di disconnettersi dal sistema.
- **RF-04 - Vista Riepilogativa Delle Issue:** Il sistema deve consentire a tutti gli utenti di visualizzare una vista riepilogativa delle issue, con possibilità di filtrare o ordinare i risultati.
- **RF-05 - Visualizzazione degli allegati:** Il sistema deve consentire a tutti gli utenti di visualizzare un'immagine allegata a una issue.
- **RF-06 - Prendi in carico:** Un utente interno può selezionare una issue nello stato "Todo" e assegnarla a sé stesso, determinandone il passaggio allo stato "Assegnato".
- **RF-07 - Rilascia issue:** Un utente interno può selezionare una issue a lui assegnata e rilasciarla, riportandola allo stato "Todo".
- **RF-08 - Segna come risolta:** Un utente interno può selezionare una issue a lui assegnata e impostarne lo stato a "Risolto".
- **RF-09 - Segnalazione di una Issue:** Un utente interno/amministratore può segnalare una nuova issue andando a specificare un titolo, una descrizione e la tipologia. È inoltre possibile specificare una priorità ed allegare un'immagine. Le issue possono essere di diverso tipo: Question, Bug, Documentation, e Feature. Appena segnalata una issue è nello stato "Todo".
- **RF-10 - Visualizzazione delle Issue archiviate:** Un utente interno può visualizzare la lista delle issue archiviate, con la possibilità di cercare o filtrare in maniera specifica.
- **RF-11 - Contrassegnazione di un issue come duplicata:** Un amministratore può contrassegnare una issue come duplicata di un'altra, al fine di evitare la presenza di issue duplicate nella lista. La contrassegnazione comporta l'eliminazione immediata della issue duplicata.

- **RF-12 - Creazione di un utente:** Un amministratore può scegliere di creare una nuova utenza, andando a specificare una e-mail, una password e un ruolo (utente interno/esterno o amministratore), Il sistema deve verificare che l'e-mail specificata non sia già associata a un account esistente.
- **RF-13 - Gestione dello stato dell'utente:** Un amministratore deve avere la possibilità di attivare o disattivare un utente interno/esterno del sistema.
- **RF-14 - Archiviazione di una issue:** Un amministratore può archiviare una issue presente nella lista delle issue attive. L'issue archiviata non deve più comparire nelle liste principali, ma deve rimanere accessibile attraverso un'apposita sezione.
- **RF-15 - Visualizzazione utenti del sistema:** Un amministratore può visualizzare una tabella contenente una lista di tutte le utenze del sistema. In particolar modo potrà visualizzarne email, ruolo e lo stato dell'account (attivo/disattivato).
- **RF-16 - Visualizzazione Dashboard:** Il sistema deve dare la possibilità ad un amministratore di visualizzare una dashboard riepilogativa. Quest'ultima deve consentire all'amministratore di visualizzare informazioni aggregate relative alle issue attive, tra cui la distribuzione per stato e tipologia, il numero di issue assegnate agli utenti e il tempo medio di risoluzione, con possibilità di individuare gli utenti caratterizzati dai minori tempi medi di risoluzione.

4.2 Requisiti non-funzionali e di sistema

BugBoard26 deve rispettare i seguenti vincoli architetturali e prestazionali:

- **Prestazioni:** Il sistema deve garantire tempi di risposta contenuti per le operazioni principali, evitando ritardi percepibili dall'utente in condizioni operative ordinarie. La generazione della dashboard deve mantenere tempi di risposta adeguati anche in presenza di un elevato numero di issue.
- **Affidabilità:** Il sistema deve garantire integrità e consistenza dei dati anche in caso di interruzioni. Devono essere previsti meccanismi di rollback e gestione eccezioni a livello di rete e applicazione.
- **Sicurezza:** Il sistema deve garantire l'autenticazione degli utenti e l'autorizzazione delle operazioni sulla base del ruolo associato all'account, impedendo l'accesso alle funzionalità non consentite. Le password devono essere composte da almeno 8 caratteri e contenere almeno una lettera maiuscola, un numero e un carattere speciale.
- **Usabilità:** L'interfaccia utente deve essere intuitiva e coerente, le funzionalità principali devono essere raggiungibili con meno interazioni possibili. L'applicazione deve essere facilmente utilizzabile anche da utenti esterni in modalità "read-only".
- **Paradigma di Programmazione:** L'implementazione deve utilizzare obbligatoriamente un linguaggio di programmazione Object-Oriented.
- **Architettura Distribuita:** Il sistema deve essere implementato prevedendo due macro-componenti separate e debolmente accoppiate: un Back-end e un Front-end.

- **Indipendenza del Front-end:** Il Front-end deve comunicare con il Back-end esclusivamente attraverso la rete, sfruttando le API esposte, e deve poter essere eseguito separatamente.
- **Gestione Centralizzata (Back-end):** Il Back-end gestisce centralmente la logica applicativa e la persistenza dei dati del sistema.
- **Manutenibilità:** Il codice sorgente deve essere commentato, modulare e aderente ai principi SOLID. Devono essere utilizzati strumenti di versioning come Git e una piattaforma di hosting del repository, come GitHub, per il tracciamento delle modifiche e per la collaborazione. È richiesto l'impiego di un CASE tool (es. Visual Paradigm) per favorire la documentazione e l'evoluzione del progetto.
- **Testabilità:** Il sistema deve consentire l'esecuzione di test automatici di unità e deve supportare l'analisi della qualità del codice tramite strumenti come ad esempio SonarQube.

5 Specifica dei Casi d'Uso

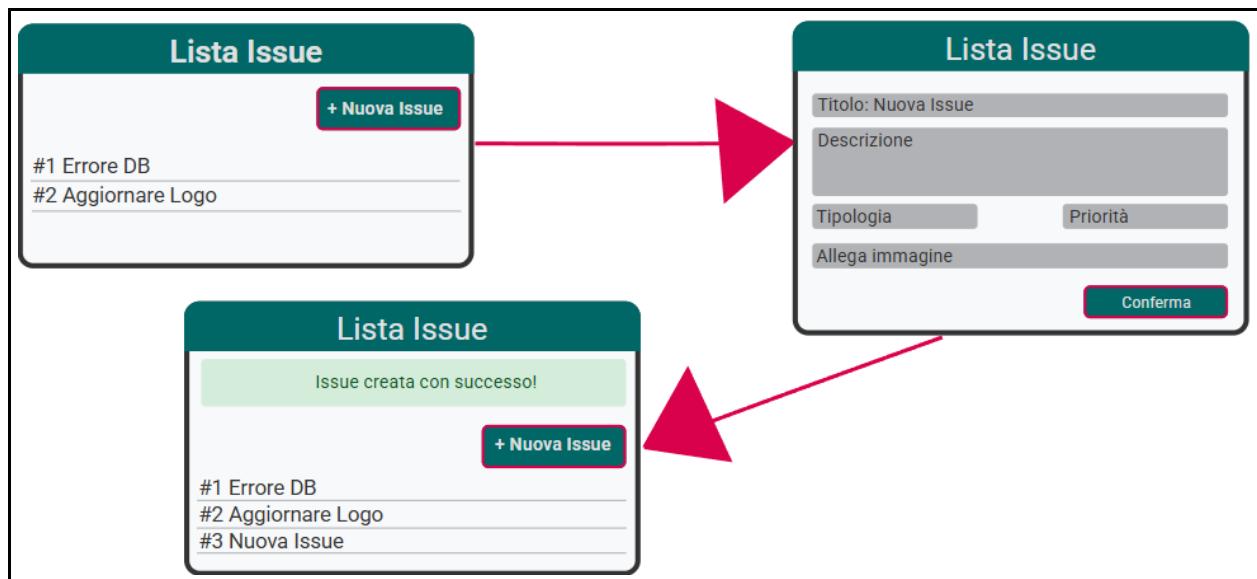
5.1 UC1: Segnalazione di una Issue

USE CASE #1	Segnalazione di una Issue		
Goal in Context	Consentire a un utente interno/amministratore di segnalare una issue.		
Preconditions	L'utente deve essere autenticato nel sistema.		
Success End Condition	La issue viene salvata nel sistema nello stato "Todo".		
Failed End Condition	La issue non viene creata e i dati inseriti non vengono salvati.		
Primary Actor	Utente interno o Amministratore		
Trigger	L'utente clicca sul pulsante "Nuova Issue".		
Main Scenario	Step n.	Actor 1 (Utente interno/admin)	System
	1		Mostra il form di inserimento per una nuova issue
	2	Inserisce titolo e descrizione	
	3	Specifica tipo e può anche specificare priorità e allegare un'immagine	
	4	Conferma la creazione	
	5		Valida i dati inseriti
	6		Salva la issue e aggiorna la lista
Extension A: Dati mancanti	5a		Segnala errore sui campi
	6a	Corregge i campi	Torna al passo 5

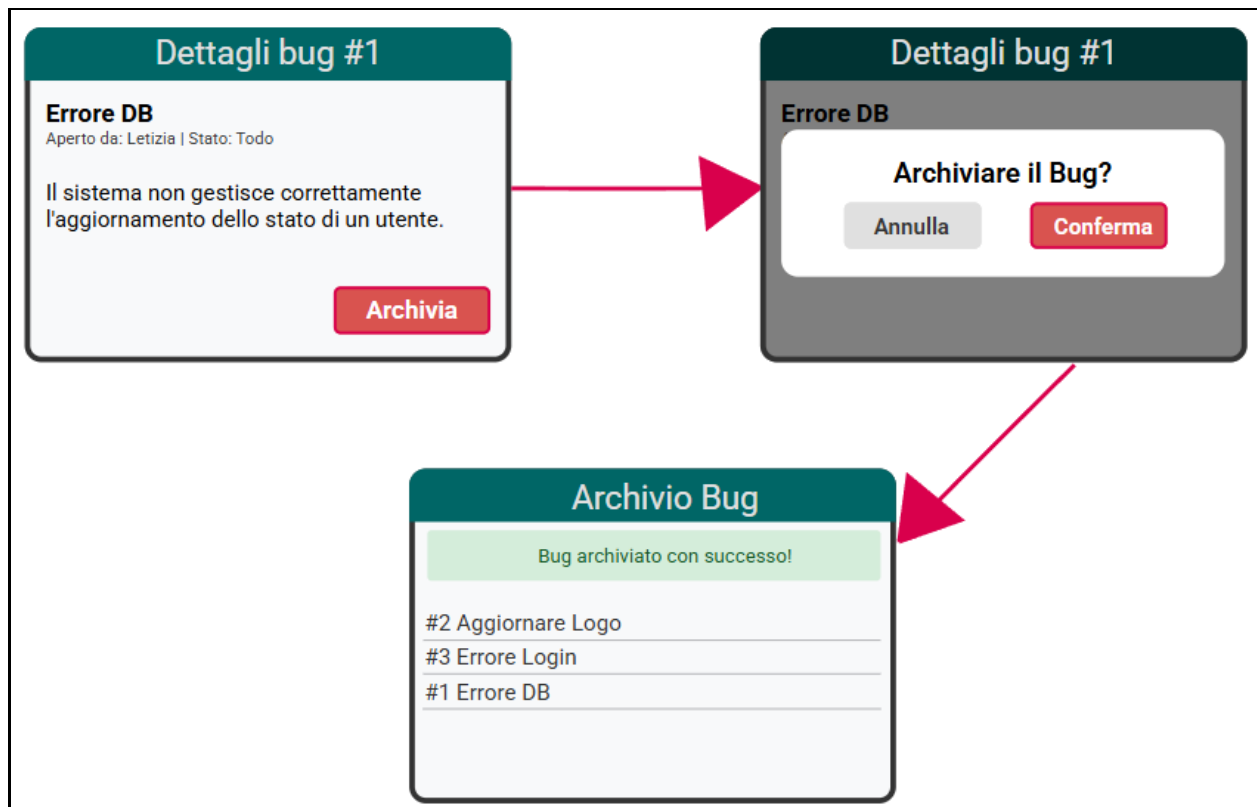
5.2 UC2: Archiviazione di una issue

USE CASE #2	Archiviazione di una Issue		
Goal in Context	Permettere a un amministratore di archiviare una issue, rimuovendola dalle liste principali.		
Preconditions	L'utente deve essere autenticato come Amministratore e la issue deve esistere.		
Success End Condition	La issue assume lo stato "Archiviato" e non è più visibile nelle liste principali, risultando consultabile nella sezione dedicata.		
Failed End Condition	Lo stato della issue rimane inalterato (es. operazione annullata).		
Primary Actor	Amministratore		
Trigger	L'amministratore seleziona l'opzione "Archivia" su una issue.		
Main Scenario	Step n.	Actor 1 (Admin)	System
	1		Chiede conferma dell'archiviazione tramite pop-up
	2	Conferma l'archiviazione	
	3		Modifica lo stato in "Archiviato"
	4		Rimuove la issue dalle liste principali
	5		La issue diventa visibile nella pagina dedicata alle issue archiviate.
	6		Mostra messaggio di successo
Extension A: Annullamento	2a	Annulla l'operazione	Chiude la finestra di conferma e annulla l'operazione

5.3 Mock-Up UC1: Segnalazione di una Issue

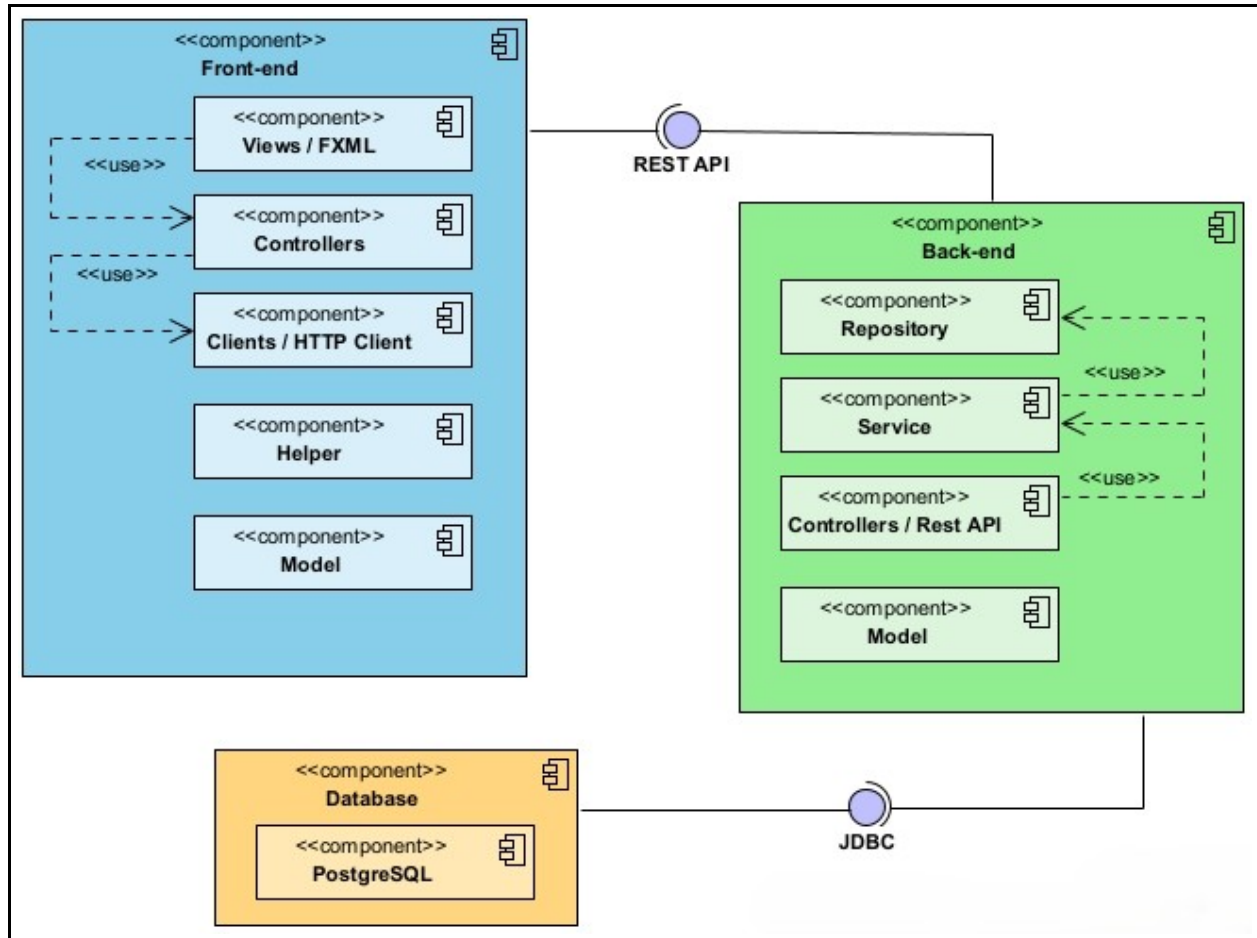


5.4 Mock-Up UC2: Archiviazione di una Issue



6 Architettura Proposta

L'architettura del sistema BugBoard26 è stata progettata secondo il paradigma Client-Server, con una netta separazione tra le due principali componenti del sistema: il Back-end, responsabile della logica applicativa e della gestione dei dati, e il Front-end, responsabile dell'interazione con l'utente. Le due componenti risultano debolmente accoppiate e comunicano attraverso le API REST esposte dal server.

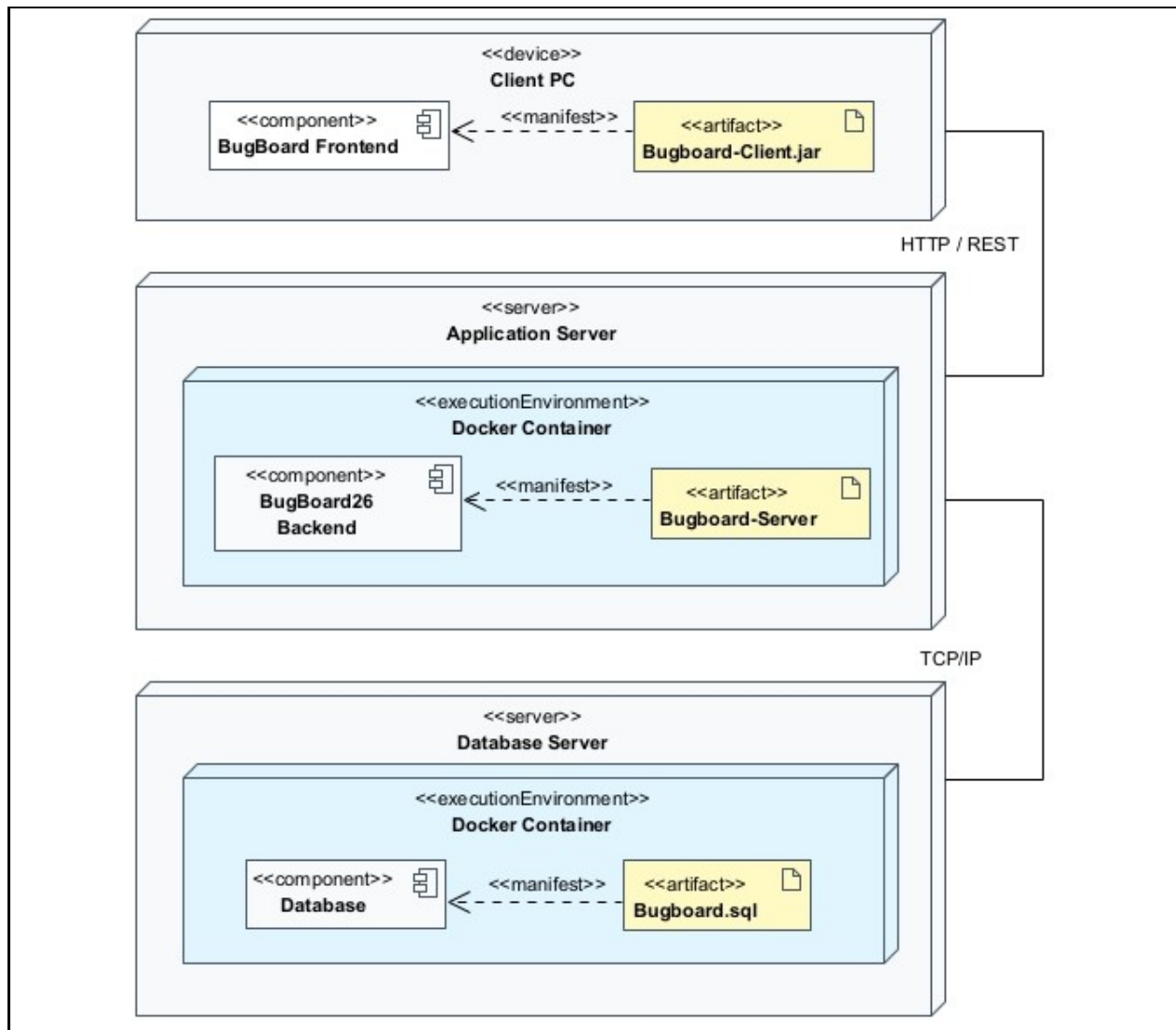


- **Flusso Front-end:** Il modulo client è strutturato secondo il pattern MVC. Le interfacce grafiche (`Views / FXML`) interagiscono con i relativi `Controllers`, responsabili della gestione degli eventi e delle azioni dell'utente. I `Controllers` delegano quindi la comunicazione con il server al modulo `Clients / HTTP Client`, incaricato di costruire e inoltrare le richieste HTTP.
- **Flusso Back-end:** Il server è organizzato secondo un'architettura a livelli. Le richieste HTTP/REST provenienti dal client vengono gestite dai `Controllers / Rest API`, che si occupano di ricevere le richieste e delegare l'elaborazione al livello `Service`. Quest'ultimo contiene la logica applicativa e le principali regole di business, utilizzando il livello `Repository` per l'accesso e la gestione dei dati persistenti.

- **Persistenza e Moduli Trasversali:** Il livello Repository interagisce con il database PostgreSQL attraverso JPA, consentendo di astrarre le operazioni di accesso e manipolazione dei dati. I moduli `Model`, presenti sia nel front-end sia nel back-end, vengono utilizzati per rappresentare i dati dell'applicazione, contribuendo alla separazione delle responsabilità e all'organizzazione dei componenti.

6.1 Deployment Diagram

Il seguente diagramma rappresenta l'infrastruttura di deployment del sistema, mostrando come i diversi artefatti software siano distribuiti tra i nodi di elaborazione, costituiti da: Client, Application Server e Database Server. È inoltre evidenziata la containerizzazione degli ambienti tramite Docker e i protocolli di rete utilizzati per la comunicazione tra i vari nodi.



6.2 Back-end (Server)

Il Back-end è stato progettato per essere indipendente dalla specifica interfaccia utente. Esso si fa carico dell'intera logica di business e della persistenza dei dati, garantendo una gestione centralizzata della logica applicativa e dei dati del sistema.

La struttura del Back-end è organizzata in componenti *Model*, *Repository*, *Service* e *Controller*, ciascuno caratterizzato da responsabilità specifiche. I *Controller* gestiscono le richieste HTTP provenienti dai client, i *Service* concentrano la logica applicativa, i *Repository* gestiscono l'accesso ai dati persistenti e i *Model* rappresentano le principali entità del dominio.

Per consentire la comunicazione con il Front-end, il Back-end espone un insieme di API REST attraverso le quali i client autorizzati possono effettuare le operazioni previste dal sistema. L'adozione di questa separazione consente inoltre di sviluppare o sostituire il client senza modificare la logica applicativa presente sul server.

Il progetto è inoltre containerizzato tramite Docker, soluzione che permette di rendere più uniforme l'ambiente di esecuzione e di semplificare le successive operazioni di deployment.

6.3 Front-end (Client)

Il Front-end rappresenta l'interfaccia attraverso cui gli utenti interagiscono con la piattaforma. Esso non contiene la logica di business principale, che rimane centralizzata nel Back-end, e comunica con il server esclusivamente attraverso le API REST esposte.

Internamente, il client è stato strutturato secondo il pattern architetturale MVC (*Model-View-Controller*). Le *View*, realizzate tramite FXML, definiscono la struttura e la presentazione delle interfacce grafiche, mentre i *Controller* gestiscono gli eventi e le interazioni dell'utente. Le operazioni di comunicazione con il Back-end sono invece demandate a un livello client dedicato. I *Model* rappresentano invece i dati utilizzati dal client e costituiscono il livello di gestione delle informazioni visualizzate nelle interfacce.

Questa organizzazione consente di mantenere separate la presentazione, la gestione degli eventi e la comunicazione con il server, favorendo un maggiore disaccoppiamento tra i diversi componenti del Front-end.

6.4 Criteri di Design e Astrazione

La scelta di adottare un'architettura distribuita Client-Server è motivata dai seguenti criteri di design:

- **Separazione delle responsabilità:** la logica di business è mantenuta separata dalla presentazione e dalle funzionalità di comunicazione, mentre la suddivisione del Back-end in livelli consente di assegnare a ciascun componente responsabilità ben definite.
- **Astrazione e Riutilizzo:** le API REST costituiscono un livello di astrazione tra client e server, permettendo di riutilizzare la logica applicativa del Back-end anche in presenza di eventuali client differenti.
- **Estensibilità:** la separazione tra Front-end e Back-end permette di modificare ed evolvere i due componenti in maniera relativamente indipendente, semplificando l'introduzione di nuove funzionalità e di eventuali ulteriori client.

7 Scelte Tecnologiche

La selezione delle tecnologie è ricaduta su strumenti consolidati e conformi al paradigma Object-Oriented. Tale scelta è stata motivata dalla necessità di realizzare un'architettura modulare e facilmente manutenibile. L'approccio orientato agli oggetti consente infatti di sfruttare principi quali incapsulamento e astrazione, favorendo il disaccoppiamento tra i componenti e il riutilizzo del codice.

7.1 Tecnologie Back-end

Per la realizzazione del Back-end, si è scelto di adottare il linguaggio **Java**. L'adozione di tale tecnologia è motivata dalla sua natura fortemente tipizzata e dal suo paradigma Object-Oriented, caratteristiche che favoriscono la robustezza e la manutenibilità del codice.

Java è stato affiancato al framework **Spring Boot**, scelto per semplificare la realizzazione delle API REST e la configurazione dell'applicazione. Il framework supporta inoltre il principio della *Dependency Injection*, consentendo di ridurre l'accoppiamento tra i componenti e di facilitarne la gestione.

La persistenza è stata implementata mediante **Spring Data JPA**, che permette di interagire con il database attraverso il mapping tra le entità Java e le tabelle relazionali. Il database utilizzato è **PostgreSQL**, un DBMS relazionale open source che consente di gestire in maniera strutturata e consistente i principali dati del dominio, tra cui utenti, issue e allegati.

La gestione dell'autenticazione e dell'autorizzazione è stata affidata a **Spring Security**, integrato con un meccanismo di autenticazione basato su **JWT**. Il Back-end verifica il token associato alle richieste ricevute e applica i relativi controlli di accesso in funzione del ruolo dell'utente.

7.2 Tecnologie Front-end

Il Front-end è stato sviluppato utilizzando **JavaFX**, piattaforma grafica per lo sviluppo di applicazioni desktop in ambiente Java. La tecnologia è stata scelta per la possibilità di realizzare interfacce grafiche desktop integrate con l'ecosistema Java.

L'utilizzo di **JavaFX** insieme al linguaggio di markup **FXML** ha permesso di strutturare il client secondo il pattern MVC, separando la definizione delle interfacce grafiche (*View*) dalla gestione degli eventi e delle interazioni (*Controller*).

La comunicazione con le API del Back-end è stata affidata a un livello **Client** dedicato, basato sulla classe **HttpClient**, supportato da apposite classi **Helper**.

Per lo scambio di dati in formato **JSON** tra Front-end e Back-end viene utilizzata la libreria **Jackson**. Essa consente di effettuare la *serializzazione* degli oggetti Java in formato JSON e la *deserializzazione* dei dati JSON negli oggetti Java corrispondenti. In questo modo, il client può convertire le risposte ricevute dalle API REST in oggetti Java e serializzare i dati da inviare al Back-end, semplificando la gestione della comunicazione tra i due componenti.

7.3 Strumenti di Sviluppo e Modellazione

Per le fasi di analisi e progettazione orientata agli oggetti è stato utilizzato il tool CASE **Visual Paradigm**, che ha consentito la realizzazione e l'esportazione dei diagrammi UML.

Per la gestione del codice sorgente e la collaborazione tra i membri del gruppo è stato utilizzato **Git**, con **GitHub** come piattaforma di hosting del repository. L'utilizzo dei branch ha consentito di sviluppare le singole funzionalità in maniera isolata, mantenendo il ramo principale del progetto in uno stato stabile.

Per il controllo della qualità del codice è stato inoltre utilizzato **SonarQube**, strumento di analisi statica che ha permesso di individuare *code smells*, potenziali bug e vulnerabilità, contribuendo all'applicazione dei principi di *Clean Code* e al miglioramento della manutenibilità del progetto.

8 Piano e Strategie di Testing

Al fine di verificare la corretta implementazione della logica di business e valutarne la robustezza, il sistema è stato sottoposto a un processo di validazione. L'attività di Testing si è concentrata sul livello Service del Back-end, isolando le dipendenze esterne, come l'accesso al database, mediante il framework Mockito.

L'approccio adottato combina tecniche di collaudo funzionale **Black-box**, basate sull'identificazione delle **classi di equivalenza** per la suddivisione degli input in categorie rappresentative e sulla verifica del comportamento del sistema rispetto agli output attesi, con tecniche di collaudo strutturale **White-box**, che tengono conto della struttura interna del codice e sono finalizzate a massimizzare la copertura delle **diramazioni condizionali (Branch Coverage)**. In questo modo, il collaudo consente di verificare sia il comportamento funzionale della logica applicativa rispetto agli input e agli output attesi, sia la corretta esecuzione dei principali percorsi logici implementati nel codice.

8.1 Test Plan: Assegnazione Issue

ID Test Plan: Assegnazione Issue

Funzionalità Target: Assegnazione di una issue ad un Internal User per la risoluzione del problema.

Obiettivo del Test: Verificare che il sistema permetta correttamente ad un utente autorizzato di prendere in carico una Issue, aggiornando lo stato di quest'ultima nel database da TO_DO ad ASSEGNATO, impostando l'id dell'assegnatario e la data di presa in carico.

Prerequisiti e Ambiente:

- Il sistema Backend (*Spring Boot*) deve essere in esecuzione.
- Devono essere disponibili i dati necessari alla configurazione dello scenario di test, tra cui un utente attivo (*Internal User*) e una Issue in stato TO_DO.
- Il test viene eseguito in isolamento (*Unit Testing*) tramite i framework JUnit 5 e Mockito, utilizzando mock delle dipendenze del Service e senza accedere al database reale.

Elementi da NON testare:

- L'interfaccia grafica non è oggetto di questo Test Plan (viene testata soltanto la logica di business del Service).
- L'autenticazione dell'utente (oggetto di un altro test).

Criteri di Superamento: Il Test Plan si considera superato se:

- Tutti gli Unit Test automatizzati vengono eseguiti con esito positivo.
- Viene raggiunta una copertura strutturale del 100% sul metodo `assegnaIssueUtente`.

8.2 Strategie di Testing

Per verificare la correttezza della logica di business è stato adottato un approccio combinato, basato sia su tecniche di collaudo funzionale sia su tecniche di collaudo strutturale. Per la realizzazione dei test sono stati utilizzati i framework JUnit 5 e Mockito, quest'ultimo impiegato per isolare le dipendenze dei componenti sottoposti a verifica. Nelle sezioni successive vengono illustrate le strategie di testing applicate a tre metodi significativi del progetto, entrambi caratterizzati dalla presenza di più parametri.

8.2.1 Analisi Metodo cambio password

La progettazione dei test per il cambio password si è basata sull'identificazione delle Classi di Equivalenza. L'obiettivo è stato coprire sia le partizioni valide che quelle non valide degli input.

Metodo: `void changePassword(String email, String oldPass, String newPass)`

Identificazione delle Classi di Equivalenza:

- **CE1 (Valida):** Email esistente, oldPass corretta, newPass diversa dalla precedente. (*Comportamento atteso: Successo e salvataggio DB*).
- **CE2 (Non Valida):** Email non presente nel DB. (*Comportamento atteso: Lancio eccezione per email non valida*).
- **CE3 (Non Valida):** Email esistente, ma oldPass non corrispondente all'hash salvato. (*Comportamento atteso: Lancio eccezione per vecchia password errata*).
- **CE4 (Non Valida):** Email esistente, oldPass corretta, ma newPass coincidente con la vecchia. (*Comportamento atteso: Lancio eccezione per password duplicata*).

Copertura Strutturale: Attraverso la configurazione dei Mock e l'esecuzione dei diversi casi di test, sono state esercitate tutte le diramazioni condizionali presenti nel metodo, raggiungendo una **Branch Coverage del 100%**.

```
public void changePassword (String email, String oldPassword, String newPassword) {  
    AuthUser user = authUserRepository.findByEmail(email).orElseThrow(() ->  
        new IllegalArgumentException("Email non valida"));  
  
    if(!passwordEncoder.matches(oldPassword,user.getPassword())) {  
        throw new IllegalArgumentException("Inserire la vecchia password corretta");  
    }  
  
    if(newPassword.equals(oldPassword)) {  
        throw new IllegalArgumentException("La nuova password non può essere uguale alla precedente");  
    }  
  
    user.setPassword(passwordEncoder.encode(newPassword));  
    authUserRepository.save(user);  
}
```

8.2.2 Analisi metodo assegnazione di una issue ad un utente

Per l'assegnazione delle issue, il metodo presenta un blocco `try-catch` interno che gestisce le eccezioni senza propagarle al chiamante, restituendo `false` in caso di errore.

Metodo: `boolean assegnaIssueUtente(int issueId, String emailUser)`

Identificazione delle Classi di Equivalenza:

- **CE1 (Valida):** `IdIssue` associato ad un issue esistente nel DB, `emailUser` associata ad un utente attivo ed esistente. (*Comportamento atteso: Return true e aggiornamento di stato e dataAssegnazione dell'issue*).
- **CE2 (Non Valida):** `IdIssue` inesistente. (*Comportamento atteso: Caduta nel blocco catch, return false, nessuna interazione con la repository degli Utenti*).
- **CE3 (Non Valida):** `IdIssue` valido, ma `emailUser` inesistente. (*Comportamento atteso: Caduta nel blocco catch, return false, nessun aggiornamento dell'issue*).

Strategia di Mocking e Verifica Comportamentale: Oltre al controllo dei valori restituiti, è stata applicata la **Behavior Verification** tramite il metodo `verify()` di Mockito. Questo ha permesso di verificare che, nelle partizioni non valide, il metodo `save()` della repository non venga invocato, impedendo l'aggiornamento dell'issue in presenza di dati non validi. Anche per questo metodo è stata raggiunta una **Branch Coverage del 100%**.

```
public boolean assegnaIssueUtente(int issueId, String emailUser) {  
    try{  
        Issue issue = issueRepository.findById(issueId)  
            .orElseThrow(() -> new RuntimeException("Issue non trovata"));  
  
        AuthUser user = authUserRepository.findByEmail(emailUser).orElseThrow(() ->  
            new RuntimeException("Utente non trovato"));  
  
        issue.setStato(StatoIssue.ASSEGNATO);  
        issue.setAssignee(user);  
        issue.setDataAssegnazione(LocalDate.now(ZoneId.systemDefault()));  
  
        issueRepository.save(issue);  
        return true;  
    } catch(Exception _){  
        return false;  
    }  
}
```

8.2.3 Analisi metodo login

La progettazione dei test per il login si è basata sull'identificazione delle Classi di Equivalenza. L'obiettivo è stato coprire sia le partizioni valide che quelle non valide degli input.

Metodo: `String login(String email, String password)`

Identificazione delle Classi di Equivalenza:

- **CE1 (Valida):** email presente nel database, password corrisponde e l'account è attivo. (*Comportamento atteso: autenticazione completata con successo e restituzione del token JWT al chiamante*).
- **CE2 (Non Valida):** email presente nel database, password corrisponde ma l'account risulta non attivo. (*Comportamento atteso: Lancio eccezione per account esistente ma non attivo*).
- **CE3 (Non Valida):** email presente nel database, ma password non corrisponde. (*Comportamento atteso: Lancio eccezione per password non valida*).
- **CE4 (Non Valida):** email non presente nel database. (*Comportamento atteso: Lancio eccezione per email non valida*).

Copertura Strutturale: Attraverso la configurazione dei Mock e l'esecuzione dei diversi casi di test, sono state esercitate tutte le diramazioni condizionali presenti nel metodo, raggiungendo una **Branch Coverage del 100%**.

```
public String login (String email, String password) {  
  
    AuthUser user = authUserRepository.findByEmail(email).orElseThrow(() ->  
        new IllegalArgumentException("Email non valida"));  
  
    if(!passwordEncoder.matches(password,user.getPassword())) {  
        throw new IllegalArgumentException("Password non valida");  
    }  
    if(!user.getStatoAccount()){  
        throw new IllegalArgumentException("Account esistente ma non attivo");  
    }  
  
    return jwtService.generateToken(user, user.getRuolo().name());  
}
```

8.3 Codice Sorgente dei Test

Di seguito viene riportato il codice sorgente relativo ai tre test di unità automatizzati descritti nella sezione precedente. Le classi di test sono state implementate in Java, avvalendosi del framework JUnit 5 e Mockito.

8.3.1 Test: Cambio Password

```
@Test
@DisplayName("Password aggiornata")
void testChangePassword_OK() {
    String email = "test1@bugboard.com";
    AuthUser user = new AuthUser();
    user.setEmail(email);
    user.setPassword("oldHashedPassword");

    when(userRepository.findByEmail(email)).thenReturn(Optional.of(user));
    when(passwordEncoder.matches("oldPassword", "oldHashedPassword"))
        .thenReturn(true);
    when(passwordEncoder.encode("newPassword"))
        .thenReturn("newHashedPassword");

    assertDoesNotThrow(() ->
        authService.changePassword(email, "oldPassword", "newPassword")
    );
    assertEquals("newHashedPassword", user.getPassword());
    verify(userRepository, times(1)).save(user);
}
```

```
@Test
@DisplayName("FAIL: Utente non trovato")
void testChangePassword_UtenteNonTrovato() {
    String email = "test2@bugboard.com";

    when(userRepository.findByEmail(email)).thenReturn(Optional.empty());

    IllegalArgumentException exc = assertThrows(
        IllegalArgumentException.class, () ->
        authService.changePassword(email, "oldPassword", "newPassword")
    );

    assertEquals("Email non valida", exc.getMessage());
}
```

```
@Test
```

```
@DisplayName("FAIL: vecchia password errata")
void testChangePassword_OldPasswordErrata() {
    String email = "test3@bugboard.com";
    AuthUser user = new AuthUser();
    user.setEmail(email);
    user.setPassword("oldHashedPassword");

    when(userRepository.findByEmail(email)).thenReturn(Optional.of(user));
    when(passwordEncoder.matches("passwordErrata", "oldHashedPassword"))
        .thenReturn(false);

    IllegalArgumentException exc = assertThrows(
        IllegalArgumentException.class, () ->
            authService.changePassword(email, "passwordErrata",
                "nuovaPassword")
    );

    assertEquals("Inserire la vecchia password corretta", exc.getMessage());
}
```

```
@Test
@DisplayName("FAIL: nuova = vecchia")
void testChangePassword_StessaPassword() {
    String email = "test4@bugboard.com";
    AuthUser user = new AuthUser();
    user.setEmail(email);
    user.setPassword("oldHashedPassword");

    when(userRepository.findByEmail(email)).thenReturn(Optional.of(user));
    when(passwordEncoder.matches("newPassword", "oldHashedPassword"))
        .thenReturn(true);

    IllegalArgumentException exc = assertThrows(
        IllegalArgumentException.class, () ->
            authService.changePassword(email, "newPassword", "newPassword")
    );

    assertEquals("La nuova password non può essere uguale alla precedente",
        exc.getMessage());
}
```

8.3.2 Test: Login

```
@Test
@DisplayName("Login effettuato con successo")
void testLogin_OK() {
    String email = "test5@bugboard.com";
    AuthUser user = new AuthUser();
    user.setEmail(email);
    user.setPassword("hashedPassword");
    user.setStatoAccount(true);
    user.setRuolo(Ruolo.INTERNAL_USER);

    when(userRepository.findByEmail(email)).thenReturn(Optional.of(user));
    when(passwordEncoder.matches("password", "hashedPassword"))
        .thenReturn(true);
    when(jwtService.generateToken(user, "INTERNAL_USER"))
        .thenReturn("token");

    String generatedToken = authUserService.login(email, "password");

    assertNotNull(generatedToken);
    assertEquals("token", generatedToken);
}

@Test
@DisplayName("FAIL: Account non attivo")
void testLogin_AccountNonAttivo() {
    String email = "test6@bugboard.com";
    AuthUser user = new AuthUser();
    user.setEmail(email);
    user.setPassword("hashedPassword");

    user.setStatoAccount(false);
    user.setRuolo(Ruolo.INTERNAL_USER);

    when(userRepository.findByEmail(email)).thenReturn(Optional.of(user));
    when(passwordEncoder.matches("password", "hashedPassword"))
        .thenReturn(true);

    IllegalArgumentException exc = assertThrows(
        IllegalArgumentException.class, () ->
            authUserService.login(email, "password"));

    assertEquals("Account esistente ma non attivo", exc.getMessage());
}
```

```
@Test
@DisplayName("FAIL: Password non valida")
void testLogin_PasswordNonValida() {
    String email = "test7@bugboard.com";
    AuthUser user = new AuthUser();
    user.setEmail(email);
    user.setPassword("hashedPassword");

    user.setStatoAccount(true);
    user.setRuolo(Ruolo.INTERNAL_USER);

    when(userRepository.findByEmail(email)).thenReturn(Optional.of(user));
    when(passwordEncoder.matches("password", "hashedPassword"))
        .thenReturn(false);

    IllegalArgumentException exc = assertThrows(
        IllegalArgumentException.class, () ->
        authUserService.login(email, "password"));

    assertEquals("Password non valida", exc.getMessage());
}

@Test
@DisplayName("FAIL: Email non presente nel DB")
void testLogin_EmailNonPresente() {
    String email = "test8@bugboard.com";
    when(userRepository.findByEmail(email)).thenReturn(Optional.empty());
    IllegalArgumentException exc = assertThrows(
        IllegalArgumentException.class, () ->
        authUserService.login(email, "password"));

    assertEquals("Email non valida", exc.getMessage());
}
```

8.3.3 Test: Assegnazione issue ad un utente

```
@Test
@DisplayName("Issue assegnata con successo")
void testAssegnaIssueUtente_Success(){

    int idIssue = 1;
    String email = "testAssegnazione@bugboard.com";

    Issue issue = new Issue();
    issue.setId(idIssue);
    issue.setStato(StatoIssue.TO_DO);

    AuthUser user = new AuthUser();
    user.setEmail(email);

    when(issueRepository.findById(idIssue)).thenReturn(Optional.of(issue));
    when(authUserRepository.findByEmail(email))
        .thenReturn(Optional.of(user));

    boolean result = issueService.assegnaIssueUtente(idIssue, email);

    assertTrue(result, "Dovrebbe restituire true in caso di successo");
    assertEquals(StatoIssue.ASSEGNATO, issue.getStato(),
        "Lo stato della issue deve passare ad ASSEGNATO");
    assertEquals(user, issue.getAssignee(),
        "L'utente assegnato deve coincidere con quello trovato");
    assertNotNull(issue.getDataAssegnazione(),
        "La data di assegnazione non deve essere nulla");

    verify(issueRepository, times(1)).save(issue);
}
```

```
@Test
@DisplayName("FAIL: Issue non trovata")
void testAssegnaIssueUtente_IssueNonTrovata() {
    int idIssue = 2;
    String email = "testAssegnazione2@bugboard.com";

    when(issueRepository.findById(idIssue)).thenReturn(Optional.empty());

    boolean result = issueService.assegnaIssueUtente(idIssue, email);

    assertFalse(result,
        "Il metodo dovrebbe restituire false se l'issue non esiste");

    verify(authUserRepository, never()).findByEmail(anyString());

    verify(issueRepository, never()).save(any(Issue.class));
}

@Test
@DisplayName("FAIL: Utente non trovato")
void testAssegnaIssueUtente_UtenteNonTrovato() {
    int issueId = 3;
    String email = "utentenontrovato@bugboard.com";

    Issue issue = new Issue();
    issue.setId(issueId);

    when(issueRepository.findById(issueId)).thenReturn(Optional.of(issue));
    when(authUserRepository.findByEmail(email))
        .thenReturn(Optional.empty());

    boolean result = issueService.assegnaIssueUtente(issueId, email);

    assertFalse(result,
        "Dovrebbe restituire false se l'utente non viene trovato");

    verify(issueRepository, never()).save(any(Issue.class));
}
```